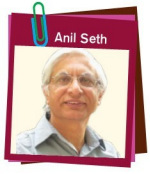

Design Apps to Use the Cores

The need for speed in computing has led to various innovations in computer design, including multicore processors. Here, the author makes a case for designing apps that will use the many cores in the processor which would otherwise remain idle. He also touches upon multiprocessing in Python.

You may have come across an announcement that Intel will soon offer desktop CPUs with eight cores. The news report also suggested that six cores may become the default minimum! The reason is simple. Increasing the speed of a CPU is now very difficult. Hence, the capacity of a system can be increased by increasing the number of cores.

However, will increasing the number of cores result in noticeably better performance? You may have a lot of applications open. There may be a lot of open windows. How many tasks in various windows are likely to be CPU bound, and will use the CPU? In all likelihood, most of the applications are waiting for you to activate their window and do something in them. Some applications or applets may occasionally wake up and check for some status or query a host.

Unless you are playing an action oriented 3D game, chances are that it will be rare to find more than a couple of cores to be active. Hence, replacing a CPU may make little difference to your desktop experience.

Just till a few years ago, the clock rate kept increasing and each application was noticeably snappier. Today, the best option for you is to replace the hard disk with a solidstate disk to notice a difference, in case you haven't already done that.

There is, however, an opportunity for developers. Various applications can be redesigned to exploit the parallel processing capabilities in the current desktops and notebooks. A great example of this approach is the Google Chrome browser. Each tab starts a separate process and, thus, actively uses the hardware to a fuller extent, provided the bandwidth and Internet connection speed are not the bottleneck. Firefox also had to follow Chrome, though limiting itself to starting a plugin as a separate process. You may notice that if you start playing a Flash video, Firefox will start an additional process. Since the desktops typically have a generous amount of RAM, it is becoming advantageous to consider moving from a multithreaded application to a multiprocessing application. Mozilla's Electrolysis project to convert Firefox to a multiprocess architecture is active again.

Even Solitaire can be made better! Many of the Solitaire games offer the hint that the game is winnable. At times, the computer has to do a lot of work before it can decide between 'Yes', 'No' and 'Don't know'. If only it would use all the cores!

Multiprocessing in Python

Creating multiple processes is fairly simple with Python. The syntax used by the multiprocessing module follows that of the threading module. The presence of the global interpreter lock in Python makes threading a harder option to increase performance in a multicore environment. The multiprocessing module makes using the cores fairly simple but at the expense of data sharing as each process has its own data.

You can create multiple processes fairly easily as the following minimal example illustrates.

```
import multiprocessing
class ExampleProcess(multiprocessing.Process):
    def run(self):
        print("Do something in " + self.name)
a = ExampleProcess()
b = ExampleProcess()
a.start()
b.start()
```

All you need to do is to create one or more classes, which inherit from *'multiprocessing.Process'* and include the method *'run'*. The harder task is to determine the following:

- What can be parallelised
- What data needs to be shared
- What data needs to be communicated between processes

As you would expect, Python's multiprocessing module makes it easy to achieve these goals after you have the design.

- You may write multiple classes, which can be run in parallel.
- You may create and start multiple process objects of the same class if the same algorithm can be run in parallel with different data.
- You may share data between processes using a Queue class or a Pipe class.
- If you must, you can share the state between processes by using shared memory, though this should be avoided.

The Python module provides a lot more functionality to manage an application designed to use the multi-process architecture. There is more than enough documentation on the Python website to explore it in detail.

A major advantage of multiprocess architecture over multithreaded architecture is reliability. A failure in a thread brings down the application while a failure in a process only halts that particular process. Hence, if you are designing a new desktop or a mobile application, or redesigning an existing one, designing it to use multiple cores may give it the extra edge over the competition. END

By: Anil Seth

The author has earned the right to do what interests him. You can find him online at *http://sethanil.com*, *http://sethanil.blogspot.com*, and reach him via email at *anil@sethanil.com*